

ACADEMIC BLOCKCHAIN CONSORTIUM

Hyperledger Fabric 2.5 • IPFS Kubo • Multi-Organization Architecture

Comprehensive Technical Architecture, System Implementation & Master Viva Voce Guide

EXECUTIVE SYSTEM DESIGN BRIEF

This technical report serves as the definitive reference specification and examination guide for the Academic Blockchain Consortium. It provides deep architectural walkthroughs of the 3-organization Hyperledger Fabric 2.5 network, off-chain IPFS storage mechanics, 7-stage cryptographic state machine, and dedicated deep-dive specifications for Contributor 5.1 (Fabric Event & History Hub Specialist) alongside 30 comprehensive Viva Voce questions and model answers.

Consortium Key Metrics & Baseline Specifications:

Component / Dimension	Specification & Configuration	Port / Topology
Blockchain Framework	Hyperledger Fabric v2.5 LTS (Enterprise Permissioned)	Raft Orderer: 7050
Consortium Organizations	Org1 (Academic Dept), Org2 (Exam Board), Org3 (Admin/Dean)	Peers: 7051, 8051, 9051
State Database	Apache CouchDB 3.3 (JSON Rich Query & Indexing)	Ports: 5984, 6984, 7984
Decentralized Storage	IPFS Kubo v0.26.0 (Cryptographic Content Addressing)	API: 5001, Gateway: 8080
Smart Contract (Chaincode)	Node.js / Fabric Contract API ("academic-contract")	Channel: academicchannel
Backend API Gateway	Express.js v4.18 REST Engine + Fabric Gateway SDK	Port: 4000
Frontend Web Portal	React 18 + Vite (SPA) + Vanilla Fallback UI	Port: 3000
Target Contributor Role	5.1: Fabric Event & History Hub Specialist (Group 5)	gRPC Listeners & History

KEY CONTRIBUTOR 5.1 FOCUS

Contributor 5.1 is the core anchor between the raw immutable ledger events and high-level client interfaces. Responsibilities include building Fabric Gateway gRPC Block & Commit Listeners, executing historical state audits via GetHistoryForKey, and serializing raw byte payloads into consumable JSON for the Conversational AI Chatbot and Ledger Explorer.

NETWORK TOPOLOGY & INFRASTRUCTURE

1. Enterprise Consortium Architecture

Detailed topology of the 3-Organization Hyperledger Fabric 2.5 Network and IPFS cluster

The Academic Blockchain Consortium is engineered as a private, permissioned distributed ledger platform governed by three autonomous organizations. Unlike public blockchains, identity in Fabric is cryptographically verified via X.509 digital certificates issued by a Membership Service Provider (MSP).

A. The Three Consortium Organizations:

- **Org 1 (Academic Department — Org1MSP):** Governs student academic intake, curriculum enrollment, faculty recommendations, Head of Department (HOD) sign-offs, and Department Academic Committee (DAC) approvals (Stages 1–4). Runs peer0.org1.academic.edu on port 7051 backed by couchdb0 on port 5984.
- **Org 2 (Examination Board — Org2MSP):** Governs exam integrity, grading, CGPA verification, academic honors classification, and transcript audits. Holds exclusive authority over the LockExamGrades smart contract transaction (Stage 5). Runs peer0.org2.academic.edu on port 8051 backed by couchdb1 on port 6984.
- **Org 3 (University Administration & Dean — Org3MSP):** Executes institutional sanction by the Dean of Academic Affairs and final credential authorization by the University Registrar (Stages 6–7 & Issuance). Anchors the certificate hash onto the ledger. Runs peer0.org3.academic.edu on port 9051 backed by couchdb2 on port 7984.

B. Consensus Mechanism & Ordering Service:

The network operates on Raft Consensus (CFT — Crash Fault Tolerant), running on orderer.academic.edu on port 7050. Raft implements a leader-follower election architecture within the channel "academicchannel". The orderer receives signed transaction proposals that have already been endorsed by the endorsing peers, sequences them into discrete chronological blocks, and distributes them to all committing peers without executing the smart contract code itself, ensuring strict deterministic execution.

C. Dual-Storage Architecture: Off-Chain IPFS + On-Chain Ledger:

A major architectural achievement of this project is the Dual-Storage Cryptographic Pattern. Raw PDF certificates (averaging 50KB to 500KB) are never stored directly within blockchain blocks to prevent catastrophic ledger state bloat. Instead:

- **Off-Chain Layer (IPFS Kubo):** The PDF document is hashed into an immutable Content Identifier (CIDv0: Qm...) and pinned to the distributed IPFS node (ports 5001/8080).
- **On-Chain Layer (Fabric Ledger):** The smart contract stores a lean, cryptographic anchor consisting of the 64-character SHA-256 document checksum, the IPFS CID, recipient student ID, degree program, and issuance timestamp under the CERT_ key namespace.

CHAINCODE STATE MACHINE

2. Cryptographic 7-Stage Endorsement Pipeline

Sequential multi-organizational state transitions enforced by chaincode business logic

Academic credentials cannot be issued unilaterally. The smart contract enforces a strict, linear 7-stage state machine where each transition requires cryptographic verification of the caller's MSP identity and current state prerequisites:

Stage #	State Identifier	Governing Org	Chaincode Function & Purpose
Stage 1	SUBMITTED	Student / Portal	CreateCertificateRequest: Generates REQ_ record with student ID & degree.
Stage 2	FACULTY_APPROVED	Org 1 (Academic)	ApproveFaculty: Validates attendance, course credits, and prerequisites.
Stage 3	HOD_APPROVED	Org 1 (Academic)	ApproveHOD: Departmental endorsement confirming completion of syllabus.
Stage 4	DAC_APPROVED	Org 1 (Academic)	ApproveDAC: Department Academic Committee verifies board clearance.
Stage 5	EXAM_LOCKED	Org 2 (Exam Board)	LockExamGrades: Locks CGPA, grade letter, honors, and comments on ledger.
Stage 6	DEAN_APPROVED	Org 3 (Admin)	ApproveDean: Academic Dean sanctions institutional graduation approval.
Stage 7	ADMIN_FINALIZED	Org 3 (Admin)	FinalizeAdmin: Registrar verifies institutional clearance; unlocks issuance.
Terminal	CERTIFICATE_ISSUED	Org 3 / Consortium	IssueCertificate: Computes SHA-256, pins to IPFS, anchors CERT_ record.

Chaincode Namespaces & World State Schema:

- **STU_{studentId}**: Stores student profile: { studentId, name, email, department, enrollmentYear, isActive }.
- **REQ_{requestId}**: Stores request state machine: { requestId, studentId, certType, status, gradesHash, history[] }.
- **CERT_{certId}**: Stores finalized anchor: { certId, requestId, studentId, studentName, department, certType, ipfsHash, docHash, issueDate }.

SPECIALIST MODULE 5.1

3. Contributor 5.1 Deep-Dive: Fabric Event & History Hub Specialist

Comprehensive analysis of gRPC listeners, GetHistoryForKey query, and ledger serialization

Contributor 5.1 occupies a pivotal engineering junction in the project. While other team members focus on submitting transactions or updating the state, Contributor 5.1 is responsible for listening to, auditing, parsing, and streaming ledger state transitions to external consumers (the AI Chatbot, UI Explorer, and Audit logs).

A. The Core Responsibilities of Contributor 5.1:

- **Blockchain Deliverable:** Implementation of Fabric Gateway gRPC Event Listeners (Block Listeners, Commit Listeners, Contract Event Listeners) and execution of GetHistoryForKey temporal queries.
- **Module / Agent Deliverable:** Serialization of low-level, binary protobuf ledger events into human-readable, schema-validated JSON data structures optimized for real-time frontend and AI agent consumption.

B. Technical Architecture of Fabric Gateway gRPC Listeners:

Traditional web applications rely on polling HTTP endpoints (request-response). In enterprise blockchain systems, polling creates massive I/O bottlenecks and induces race conditions. Contributor 5.1 implements persistent HTTP/2 gRPC event streams:

- **1. Block Listeners (`network.newBlockListener()`):** Subscribes directly to peer block-committal streams. When the Raft orderer creates a block and the peer commits it to its local ledger, a block event fires. Contributor 5.1 extracts block height, transaction count, block header hash, and transaction metadata.
- **2. Commit Listeners (`network.newCommitListener()`):** Tracks individual transaction lifecycle states. Allows the backend to know with 100% cryptographic certainty whether a transaction proposal was successfully committed, aborted due to MVCC (Multi-Version Concurrency Control) conflict, or rejected by an endorsement policy.
- **3. Contract Event Listeners (`contract.newEventListener()`):** Listens for custom events emitted by chaincode using `ctx.stub.setEvent("CertificateIssued", payload)`. Fires instantly across client applications upon credential issuance.

C. Temporal Auditing via GetHistoryForKey:

Hyperledger Fabric separates World State (current values in CouchDB) from Blockchain History (append-only ledger stored in LevelDB). While CouchDB only stores the latest status (e.g., `ADMIN_FINALIZED`), `GetHistoryForKey(key)` traverses the chronological cryptographic log to return every mutation that ever touched that key.

- **Historical Data Elements:** Each entry contains: `txId` (the unique 64-char transaction hash), `value` (the exact payload at that point in time), `timestamp` (deterministic block timestamp), and `isDelete` (boolean tombstone flag).
- **Serialization Challenge Solved by 5.1:** Raw Fabric timestamps are serialized as Google Protobuf Timestamp objects (`{ seconds: Long, nanos: Int }`). Contributor 5.1 converts these into ISO-8601 strings, decodes raw byte buffers into UTF-8 JSON, and structures the audit trail into an ordered array of historical transitions.

VIVA VOCE PREPARATION: PART 1

4. Master Viva Voce Questions & Answers — General Architecture

Comprehensive questions covering Hyperledger Fabric, Consensus, IPFS, and Cryptography

Q1: What is the fundamental difference between Hyperledger Fabric and public blockchains like Ethereum or Bitcoin?

Ans: Hyperledger Fabric is a permissioned, private enterprise blockchain framework. In Ethereum/Bitcoin, anyone can join anonymously (permissionless), transactions are public, consensus requires proof-of-work/stake with high energy/gas costs, and execution is slow. Fabric requires cryptographic identity via X.509 certificates (MSP), supports private data collections, features zero gas fees, uses high-throughput crash-fault-tolerant consensus (Raft), and executes smart contracts in standard languages like Node.js, Go, or Java.

Q2: Explain the Execute-Order-Validate (E-O-V) architecture in Fabric.

Ans: Traditional blockchains use Order-Execute (transactions are ordered into blocks, then executed by all nodes). Fabric uses Execute-Order-Validate: First, client sends proposals to endorsing peers who execute chaincode in simulation and generate Read-Write Sets (Execute). Second, endorsed proposals are sent to the Raft ordering service which packages them into blocks chronologically (Order). Third, committing peers verify signatures, check endorsement policies, and validate Read-Write sets for MVCC conflicts before committing to the ledger (Validate). This eliminates non-deterministic execution.

Q3: Why does this project use CouchDB instead of the default LevelDB for state storage?

Ans: LevelDB only supports simple key-value lookups (GetState, PutState by exact key). CouchDB is a document-oriented database that supports JSON formatting and rich query mechanisms (Mango queries). In our academic consortium, CouchDB allows complex queries like searching all students by department, querying all certificates issued in 2026, or filtering requests by status, which would be impossible with LevelDB without scanning the entire database.

Q4: Why are PDF certificates stored on IPFS rather than directly on the Hyperledger Fabric ledger?

Ans: Storing large binary files (PDFs, images) directly on blockchain blocks causes state bloat. Every peer must maintain a full copy of the blockchain history; storing megabytes of PDF data per student would cause peer storage to explode and drastically degrade block propagation latency. Instead, we use IPFS as an off-chain decentralized storage layer to hold the PDF and store only the lean 64-character SHA-256 hash and IPFS CID on the ledger.

Q5: What is the role of the Raft ordering service in our 3-organization consortium?

Ans: Raft is a Crash Fault Tolerant (CFT) ordering service. It ensures all three organizations agree on the exact chronological sequence of transactions. Raft elects a leader node that accepts endorsed transaction proposals, orders them into discrete blocks, and broadcasts the blocks to peers in Org1, Org2, and Org3. It guarantees that even if a peer fails, the network reaches consistent consensus as long as a quorum of orderers remains operational.

Q6: What is an Endorsement Policy and how is it enforced?

Ans: An endorsement policy defines which organizations must cryptographically sign a transaction before it is considered valid. For critical operations like issuing a certificate, the policy can mandate AND('Org1MSP:peer', 'Org2MSP:peer', 'Org3MSP:peer'). When committing a block, each peer validates that the transaction contains valid signatures from the required MSPs. If signatures are missing, the peer marks the transaction as INVALID.

Q7: What is Multi-Version Concurrency Control (MVCC) in Hyperledger Fabric?

Ans: MVCC prevents double-spending and state conflicts. During the execution phase, the peer records the version number of all keys read (ReadSet). During the validation phase, the peer verifies whether the version in the ReadSet matches the current version in the world state. If another transaction modified that key in the interim, an MVCC_READ_CONFLICT occurs, and the transaction is aborted.

Q8: How does the QR code on the certificate verify authenticity without internet access to the original server?

Ans: The QR code encodes a signed cryptographic payload containing: Cert ID, Student ID, SHA-256 hash, IPFS CID, and issue date. A verifier scans the QR code and re-computes the SHA-256 hash of the student's digital or printed document. If the computed hash matches the hash inside the QR code and verified against any connected blockchain peer, authenticity is mathematically proven.

Q9: What happens if someone tampers with a single character in the issued PDF certificate?

Ans: Cryptographic hash functions like SHA-256 possess the avalanche effect: changing even a single bit in the PDF completely changes the resulting 64-character hexadecimal digest. When the verification portal re-computes the hash of the tampered PDF, it will not match the immutable hash stored on the Fabric blockchain, immediately flagging the certificate as forged.

Q10: Explain the role of cryptogen and crypto-config in our network.

Ans: cryptogen is a Fabric utility that generates the cryptographic public key infrastructure (PKI) based on crypto-config.yaml. It creates root Certificate Authorities (CAs), signing certificates, TLS certificates, private keys, and MSP folder structures for OrdererOrg, Org1MSP, Org2MSP, and Org3MSP, ensuring secure mutual TLS (mTLS) across all Docker containers.

Q11: What is the difference between World State and Blockchain History?

Ans: World State represents the latest current values of all ledger keys, stored in CouchDB for rapid retrieval. Blockchain History is the immutable, append-only log of all transaction blocks stored in internal peer LevelDB. While World State changes with every update, Blockchain History never changes and records every historical state transition.

Q12: How does the REST API backend communicate with the Fabric peers?

Ans: The Express.js backend utilizes the official @hyperledger/fabric-gateway Node.js SDK. It establishes a secure gRPC connection over TLS using the client's identity certificate and private key. It calls network.getContract("academic-contract") and executes contract.submitTransaction() for ledger writes or contract.evaluateTransaction() for ledger reads.

Q13: Why does the Exam Board have exclusive rights to Stage 5 (LockExamGrades)?

Ans: Under university governance, academic departments teach and recommend students, but only the autonomous Examination Board (Org 2) has legal authority to audit examination transcripts, certify CGPA, and award honors classifications. Restricting LockExamGrades to Org2MSP enforces institutional separation of powers on the smart contract level.

Q14: How does the IPFS proxy route in the backend solve Kubo subdomain redirect issues?

Ans: Modern IPFS Kubo nodes redirect requests to subdomains (e.g. cid.ipfs.localhost:8080) for origin isolation. In local Docker environments, browsers cannot resolve dynamic subdomains. The backend implements a proxy endpoint (/api/ipfs/cid) that fetches raw bytes directly from the IPFS RPC port (ipfs-node:5001/api/v0/cat?arg=cid) and streams them with proper application/pdf headers.

Q15: What is a Channel in Hyperledger Fabric?

Ans: A channel is a private subnet of communication between specific consortium members. In our project, "academicchannel" connects Org1, Org2, and Org3. Transactions executed on a channel are completely invisible and inaccessible to peers not joined to that channel, providing enterprise data isolation.

VIVA VOCE PREPARATION: PART 2 (CONTRIBUTOR 5.1)

5. Dedicated Viva Voce Questions & Answers — Contributor 5.1 Focus

Deep-dive examination questions tailored specifically for the Fabric Event & History Hub Specialist

Q16: What exactly was your individual role as Contributor 5.1 in this consortium project?

Ans: As Contributor 5.1 (Fabric Event & History Hub Specialist), I was responsible for bridging the low-level immutable blockchain state with real-time application layers. Specifically, I engineered Fabric Gateway gRPC event listeners (Block and Commit listeners) for live ledger synchronization, implemented the GetHistoryForKey chaincode invocation for temporal auditing, and built the serialization engine that transforms raw protobuf byte streams into structured JSON for the Conversational AI Chatbot and UI Explorer.

Q17: How does GetHistoryForKey work internally in Hyperledger Fabric?

Ans: GetHistoryForKey is a native Fabric chaincode shim method (`stub.getHistoryForKey(key)`). Fabric peers maintain an internal History Database indexed by key in LevelDB (separate from CouchDB state). When called, it returns a HistoryQueryIterator that traverses the append-only block storage to retrieve every transaction ID, value payload, timestamp, and deletion status that ever modified that specific key.

Q18: Why can CouchDB NOT be used directly to query the history of a key?

Ans: CouchDB only maintains the current World State. When a key is updated, CouchDB overwrites the previous JSON document with the new revision. It has no index or awareness of older revisions or historical blocks. Therefore, historical audits must query Fabric's internal historical index (LevelDB) via `stub.getHistoryForKey()`.

Q19: What is the structure of a raw HistoricalUpdate object returned by Fabric?

Ans: Each historical record contains four fundamental fields: 1) txId: The unique 64-character SHA-256 transaction identifier. 2) value: A raw Uint8Array Buffer containing the JSON string of the state at that point. 3) timestamp: A protobuf Timestamp object containing seconds (int64) and nanos (int32). 4) isDelete: A boolean indicating whether this transaction deleted the key.

Q20: How do you handle Google Protobuf Timestamps and BigInt serialization in Node.js?

Ans: Fabric timestamps return seconds as a 64-bit Long/BigInt object. Standard `JSON.stringify()` throws a `TypeError: Do not know how to serialize a BigInt`. In my serialization pipeline, I extract the seconds property, convert it to a standard JavaScript Number using `(seconds.toNumber() * 1000 + nanos / 1e6)`, and instantiate a JavaScript Date object, outputting an ISO-8601 string (e.g., 2026-09-29T18:31:07Z).

Q21: What is the difference between a Block Listener, a Commit Listener, and a Contract Event Listener?

Ans: 1) Block Listener: Fires whenever a new block is committed to the peer; useful for global metrics like block height, transaction counts, and consortium ledger activity. 2) Commit Listener: Fires when a specific submitted transaction proposal is committed or rejected; provides transactional finality feedback to the client. 3) Contract Event Listener: Fires when chaincode explicitly invokes `ctx.stub.setEvent("EventName", payload)`; used for domain-specific events like "CertificateIssued".

Q22: Why did you choose gRPC event listeners instead of periodic HTTP polling?

Ans: HTTP polling introduces a trade-off between latency and server overhead: polling every second wastes CPU cycles and network bandwidth, while polling every 10 seconds causes UI lag. Fabric Gateway gRPC listeners establish a persistent HTTP/2 bidirectional stream. When a block commits, the peer pushes the event instantly (sub-millisecond latency) with minimal network overhead.

Q23: How does your 5.1 module feed into Contributor 5.2 (NLP Intent Engine) and 5.3 (Chatbot Widget)?

Ans: Contributor 5.2 builds the NLP intent engine that parses natural language questions like "Who approved REQ-2024-001 at HOD stage?". When 5.2 identifies the intent, it invokes my 5.1 serialization hub to fetch the full GetHistoryForKey timeline. The chatbot then parses my structured history array to respond: "HOD approval was endorsed by FacultyHead-Org1 on Sept 29 at 18:30 with comments: HOD verified". Contributor 5.3 uses this same stream to update the chat UI in real time.

Q24: What happens if an event listener disconnects due to a temporary network blip or peer restart?

Ans: Fabric Gateway v2.5 listeners support checkpointing and start block positioning. When instantiating `network.newBlockListener({ startBlock: lastSeenBlock + 1n })`, the client tells the peer where it left off. Upon reconnection, the peer replays all missed blocks sequentially, ensuring zero data loss and flawless audit synchronization.

Q25: Explain how you reconstruct the 7-stage lifecycle of a request from its raw history entries.

Ans: When a student request progresses through the 7 stages, each stage executes a `PutState(REQ_id, updatedRequest)`. By calling `GetHistoryForKey(REQ_id)`, I receive an array of historical snapshots. I iterate through each snapshot, decode the JSON payload, extract the status field (SUBMITTED -> FACULTY_APPROVED -> ... -> ADMIN_FINALIZED), and map each stage to its endorsing MSP identity, transaction hash, and timestamp to produce a complete linear audit log.

Q26: How do you distinguish between a valid transaction and an invalid transaction when listening to blocks?

Ans: Inside a Fabric block, each transaction is paired with a validation code in the block metadata (`TxValidationCode`). The peer validates transactions before committal. If a transaction has code 0 (VALID), it modified the world state. If it has code 11 (MVCC_READ_CONFLICT) or 10 (BAD_ENDORSEMENT_POLICY), it was rejected. My event listener inspects this metadata flag so the system never reports an invalid transaction as valid.

Q27: What is the format of the transaction ID generated by Fabric?

Ans: The `txId` is a 64-character hexadecimal SHA-256 hash. It is computed deterministically by hashing the serialized identity of the submitting client (MSP ID + X.509 certificate) concatenated with a cryptographically secure random nonce generated during proposal creation. This guarantees universal uniqueness across the consortium.

Q28: Can a deleted record still be audited using your module?

Ans: Yes. In Fabric, calling `ctx.stub.deleteState(key)` does NOT purge the record from the blockchain; it merely places a tombstone (`isDelete: true`) in the current World State (CouchDB). My `GetHistoryForKey` implementation retrieves all historical values prior to deletion and identifies the exact transaction and identity that executed the deletion.

Q29: What security precautions are implemented in the history query endpoint?

Ans: The `GetHistoryForKey` query requires the client to authenticate with a valid Org MSP certificate. Access control rules ensure students can only query their own request history, while departmental auditors and exam board officers can query academic records within their organizational jurisdiction, preventing unauthorized cross-department data leakage.

Q30: If an examiner asks you to demonstrate your 5.1 deliverable right now, what would you show?

Ans: I would show: 1) The `GetHistoryForKey` API endpoint returning the full chronological JSON audit trail of any request ID, showing every stage mutation, timestamp, and endorsing identity. 2) The live Ledger Explorer where full 64-character transaction hashes and IPFS CIDs are streamed. 3) The automated event synchronization showing instant UI updates when a stage is approved without manual page reloads.